



Database Management System (303105203)

Unit – 7: Query Processing and Optimization

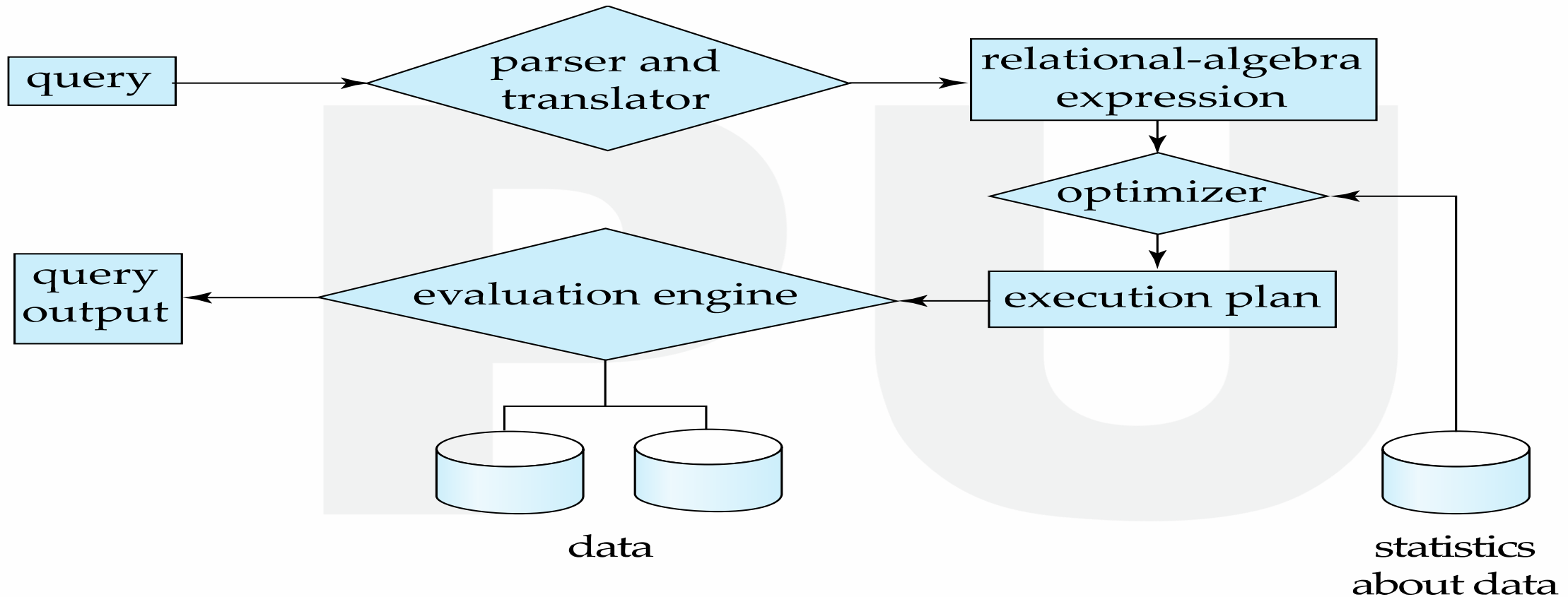
Dr. Dhanraj Verma,
Professor, CSE-AIML



Query Processing

- **Query processing** refers to the range of activities involved in extracting data from a database.
- Basic steps of query processing are:
 1. Parsing and translation
 2. Optimization
 3. Evaluation

Steps of Query processing:



Steps of Query processing:

1. Parsing and translation.

- The first action the system must take in query processing is to translate a given query into its internal form.
- This translation process is similar to the work performed by the **parser** of a compiler.
- In generating the internal form of the query, the parser checks the syntax of the user's query, verifies that the relation names appearing in the query are names of the relations in the database, and so on.
- The system constructs a parse-tree representation of the query, which it then translates into a relational-algebra expression. If the query was expressed in terms of a view, the translation phase also replaces all uses of the view by the relational-algebra expression that defines the view.

Steps of Query processing:

2. Evaluation Plan.

- A relational algebra operation annotated with instructions on how to evaluate it is called an evaluation primitive. A sequence of primitive operations that can be used to evaluate a query is a query-execution plan or **query-evaluation** plan.
- The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.

Steps of Query processing:

3. Optimization.

- It is the responsibility of the system to construct a query evaluation plan that minimizes the cost of query evaluation; this task is called **query optimization**.
- Once the query plan is chosen, the query is evaluated with that plan, and the result of the query is output.
- In order to optimize a query, a query optimizer must know the cost of each operation.

Measures of Query cost

- There are multiple possible evaluation plans for a query, and it is important to be able to compare the alternatives in terms of their (estimated) cost, and choose the best plan.
- We must estimate the cost of individual operations, and combine them to get the cost of a query evaluation plan.
- The cost of query evaluation can be measured in terms of a number of different resources, including disk accesses, CPU time to execute a query, and, in a distributed or parallel database system, the cost of network communication.
- For simplicity we just use the **number of block transfers** *from disk* and the **number of seeks** as the cost measures

t_T – time to transfer one block

t_S – time for one seek

Cost for b block transfers plus S seeks

$$b * t_T + S * t_S$$

Measures of Query cost

- The costs of all the algorithms that we consider depend on the size of the buffer in main memory.
- The **response time** for a query-evaluation plan, assuming no other activity is going on in the computer, would account for all these costs, and could be used as a measure of the cost of the plan.
- The response time of a plan is very hard to estimate without actually executing the plan, for the following reasons:
 1. The response time depends on the contents of the buffer when the query begins execution; this information is not available when the query is optimized, and is hard to account for even if it were available.
 2. In a system with multiple disks, the response time depends on how accesses are distributed among disks, which is hard to estimate without detailed knowledge of data layout on disk.
- Interestingly, a plan may get a better response time at the cost of extra resource consumption.

Selection Operation

- In query processing, the file scan is the lowest-level operator to access data.
- **File scans** are search algorithms that locate and retrieve records that fulfill a selection condition.
- In relational systems, a file scan allows an entire relation to be read in those cases where the relation is stored in a single, dedicated file.
- There are two scan algorithms to implement in selection operation.
 1. Linear search
 2. Binary search

1. Linear search (brute force algorithm)

- In a linear search(A1 Algoritm), the system scans each file block and tests all records to see whether they satisfy the selection condition. An initial seek is required to access the first block of the file.
- Since the records are grouped into disk blocks, each disk block is read into a main memory buffer, and then a search through the records within the disk block is conducted in main memory.
- Suppose t_s is the seek time (number of Seek is usually one – to reach the beginning of the file) , t_T is the number of traversal time for one block, and B is the number of blocks to be transferred, then the cost is calculated as: $tS + (B*tT)$
- This is the cost required to fetch the record based on non-key attribute. Suppose the search is based on the key value. Then the average cost of query is $tS + (B*tT)/2$.
- Although it may be slower than other algorithms for implementing selection, the linear-search algorithm can be applied to any file, regardless of the ordering of the file, or the availability of indices, or the nature of the selection operation.

2. Binary search

- If the selection condition involves an equality comparison on a key attribute on which the file is ordered, binary search which is more efficient than linear search can be used.
- This method of selection is applicable only when the records are sorted based on the search key value and we have equal condition. i.e.; this method is not suitable for range operation or any other kind. The filter condition should be 'search key column = value', like we had 'CLASS_NAME = 'DESIGN_01'.
- Suppose blocks of records are stored continuously in the memory then the cost of the query to fetch first record is calculated as $\log(B) * (t_s + t_T)$.
- This is because we need to traverse for each record in a tree till child node. i.e.; suppose we need to fetch two records DESIGN_01 and DESIGN_02. They are at two different child nodes. Hence, we have seek-time for each of them and transfer time for each of the nodes.

Join operations.

JOIN operation is one of the most time-consuming operations in query processing.

Equi-join is a join of the form $r \bowtie r.A=s.B$ s , where A and B are attributes or sets of attributes of relations r and s , respectively.

➤ **Nested loop join:**

Relation r is called the **outer relation** and relation s the **inner relation** of the join, since the loop for r encloses the loop for s . The algorithm uses the notation $tr \cdot ts$, where tr and ts are tuples; $tr \cdot ts$ denotes the tuple constructed by concatenating the attribute values of tuples tr and ts .

for each tuple tr **in** r **do begin**

for each tuple ts **in** s **do begin**

 test pair (tr, ts) to see if they satisfy the join condition

 if they do, add $tr \cdot ts$ to the result;

end

end

Join operations.

- **Indexed nested-loop join**
- In a nested-loop join, if an index is available on the inner loop's join attribute, index lookups can replace file scans. For each tuple tr in the outer relation r , the index is used to look up tuples in s that will satisfy the join condition with tuple tr .
- This join method is called an **indexed nested-loop join**; Looking up tuples in s that will satisfy the join conditions with a given tuple tr is essentially a selection on s .
- For example, consider $student \bowtie takes$. Suppose
- that we have a $student$ tuple with ID "00128". Then, the relevant tuples in $takes$ are those that satisfy the selection " $ID = 00128$ ".

Evaluation of Expressions

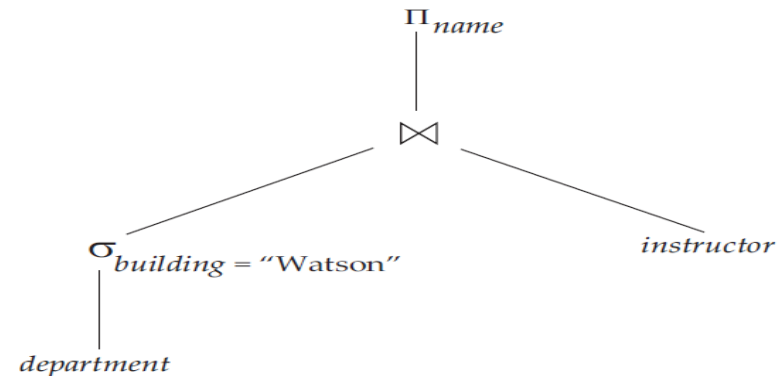
- Now we consider how to evaluate an expression containing multiple operations.
- The obvious way to evaluate an expression is simply to evaluate one operation at a time, in an appropriate order.
- There are two approaches of query optimization:-
 1. Materialization
 2. Pipeline

Evaluation of Expressions

- Now we consider how to evaluate an expression containing multiple operations.
- The obvious way to evaluate an expression is simply to evaluate one operation at a time, in an appropriate order.
- There are two approaches of query optimization:-
 1. Materialization: Generate results of an expression whose inputs are relations or are already computed, **materialize** (store) it on disk. Repeat.
 2. Pipeline: Pass on tuples to parent operations even as an operation is being executed.

Evaluation of Expressions

- 1. Materialized:
- It is easiest to understand intuitively how to evaluate an expression by looking at a pictorial representation of the expression in an **operator tree**. Consider the expression:
- $\pi_{name}(\sigma_{building = \text{"Watson"}}(department) \bowtie instructor)$



- By repeating the process, we will eventually evaluate the operation at the root of the tree, giving the final result of the expression. In our example, we get the final result by executing the projection operation at the root of the tree, using as input the temporary relation created by the join.
- Evaluation as just described is called **materialized evaluation**, since the results of each intermediate operation are created (materialized) and then are used for evaluation of the next-level operations.

Evaluation of Expressions

- **Pipelined evaluation** : evaluate several operations simultaneously, passing the results of one operation on to the next.
- E.g., in previous expression tree, don't store result of $\sigma_{building="Watson"}(department)$
 - instead, pass tuples directly to the join.. Similarly, don't store result of join, pass tuples directly to projection.
- Much cheaper than materialization: no need to store a temporary relation to disk.
- Pipelining may not always be possible – e.g., sort, hash-join.
- For pipelining to be effective, use evaluation algorithms that generate output tuples even as tuples are received for inputs to the operation.
- Pipelines can be executed in two ways: **demand driven** and **producer driven**

Evaluation of Expressions

1. In a **demand-driven pipeline**, the system makes repeated requests for tuples from the operation at the top of the pipeline. Each time that an operation receives a request for tuples, it computes the next tuple (or tuples) to be returned, and then returns that tuple.
 - If the inputs of the operation are not pipelined, the next tuple(s) to be returned can be computed from the input relations, while the system keeps track of what has been returned so far.
 - If it has some pipelined inputs, the operation also makes requests for tuples from its pipelined inputs. Using the tuples received from its pipelined inputs, the operation computes tuples for its output, and passes them up to its parent.
 - Each operation in a demand-driven pipeline can be implemented as an **iterator** that provides the following functions: *open()*, *next()*, and *close()*.



Evaluation of Expressions

2. In a **producer-driven pipeline**, operations do not wait for requests to produce tuples, but instead generate the tuples **eagerly**.

- Each operation in a producer-driven pipeline is modeled as a separate process or thread within the system that takes a stream of tuples from its pipelined inputs and generates a stream of tuples for its output.
- For each pair of adjacent operations in a producer-driven pipeline, the system creates a buffer to hold tuples being passed from one operation to the next.
- The processes or threads corresponding to different operations execute concurrently.
- Each operation at the bottom of a pipeline continually generates output tuples, and puts them in its output buffer, until the buffer is full.

Evaluation of Expressions

- An operation at any other level of a pipeline generates output tuples when it gets input tuples from lower down in the pipeline, until its output buffer is full.
- Once the operation uses a tuple from a pipelined input, it removes the tuple from its input buffer. In either case, once the output buffer is full, the operation waits until its parent operation removes tuples from the buffer, so that the buffer has space for more tuples. At this point, the operation generates more tuples, until the buffer is full again.
- The operation repeats this process until all the output tuples have been generated.

Query Optimization

- **Query optimization** is the process of selecting the most efficient query-evaluation plan from among the many strategies usually possible for processing a given query, especially if the query is complex.
- One aspect of optimization occurs at the relational-algebra level, where the system attempts to find an expression that is equivalent to the given expression, but more efficient to execute.
- When any query constructs the large intermediate relation and we want the same result but with small intermediate representation we convert that query by equivalent expression.

$$\Pi_{name, title} (\sigma_{dept_name = \text{"Music"}} (instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$$

Equivalence rule:

- ***Transformation of relational expression to equivalent relational expression.***
- A query can be expressed in several different ways, with different costs of evaluation.
- Two relational-algebra expressions are said to be **equivalent** if, on every legal database instance, the two expressions generate the same set of tuples.
- An **equivalence rule** says that expressions of two forms are equivalent. We can replace an expression of the first form by an expression of the second form, or vice versa since the two expressions generate the same result on any valid database.
- We use θ , θ_1 , θ_2 , and so on to denote tables,
- L_1 , L_2 , L_3 , and so on to denote lists of attributes,
- E , E_1 , E_2 , and so on to denote relational-algebra expressions.

Equivalence rule:

Rule:1

Conjunctive selection operations can be deconstructed into a sequence of individual selections. This transformation is referred to as a cascade of σ .

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

Rule:2

Selection operations are **commutative**.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

Rule:3

Only the final operations in a sequence of projection operations are needed; the others can be omitted. This transformation can also be referred to as a cascade of π .

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$$

Equivalence rule:

Rule: 4

Selections can be combined with Cartesian products and theta joins.

$$\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$$

Rule: 5

Theta-join operations are commutative.

$$\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$$

Rule: 6

A) Natural-join operations are **associative**.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

B) Theta joins are associative in the following manner

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

Equivalence rule:

Rule-7

The selection operation distributes over the theta-join operation under the following two conditions:

- A. It distributes when all the attributes in selection condition θ_0 involve only the attributes of one of the expressions (say, E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

- B. It distributes when selection condition θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

Rule-8

The set operations union and intersection are commutative.

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

Set difference is not commutative.

Equivalence rule:

- Rule-9
- Set union and intersection are associative.
$$(E1 \cup E2) \cup E3 = E1 \cup (E2 \cup E3)$$
$$(E1 \cap E2) \cap E3 = E1 \cap (E2 \cap E3)$$
- Rule-10
- The projection operation distributes over the union operation.
$$\pi_L (E1 \cup E2) = (\pi_L (E1)) \cup (\pi_L (E2))$$

Cost based query optimization

- A cost-based optimizer will look at all of the possible ways or scenarios in which a query can be executed – and each scenario will be assigned a ‘cost’, which indicates how efficiently that query can run.
- The database optimizes each SQL statement based on statistics collected about the accessed data.
- The optimizer determines the optimal plan for a SQL statement by examining multiple access methods, such as full table scan or index scans, different join methods such as nested loops and hash joins, different join orders, and possible transformations.
- For a given query and environment, the optimizer assigns a relative numerical cost to each step of a possible plan, and then factors these values together to generate an overall cost estimate for the plan.
- After calculating the costs of alternative plans, the optimizer chooses the plan with the lowest cost estimate.

× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.in